

CAKE SHOP

FULLY PRODUCTION-READY E-COMMERCE DATABASE SCHEMA

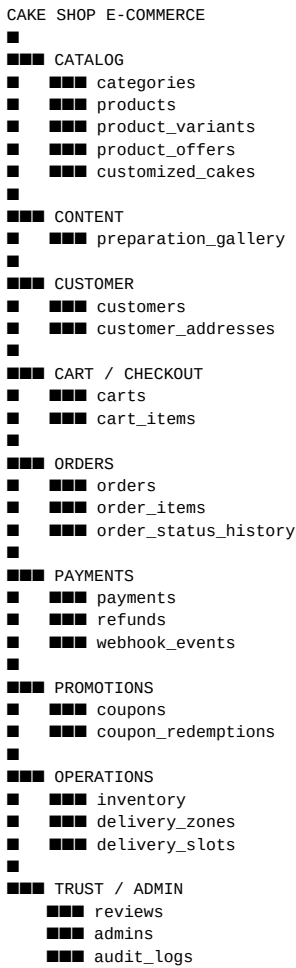
PostgreSQL (Neon) + Prisma ORM + Razorpay

Complete database architecture for catalog, variants, customized cakes, media/gallery, cart/checkout, customers, addresses, orders, payments, refunds, webhooks, coupons, delivery, reviews, inventory, audit/security, and production operations.

Area	Final decision
Database	PostgreSQL on Neon
ORM	Prisma
Payments	Razorpay with dedicated payment/refund/webhook records
Checkout	Guest checkout supported; account optional
Media	External object/CDN storage; DB stores references
Orders	Immutable historical snapshots + controlled status updates
Production	Migration-based, indexed, transactional, auditable

Scope note: This document expands the previously supplied project schemas into a complete production e-commerce database. Where a new production feature was not present in the supplied documents, it is explicitly treated as a production recommendation rather than as an existing project requirement.

1. Final Architecture



1.1 Core relationship map

Parent	Child	Relation	Key
categories	products	1:N	categories.id → products.category_id
products	product_variants	1:N	products.id → variants.product_id
products	product_offers	1:N	products.id → offers.product_id
orders	order_items	1:N	orders.id → order_items.order_id
orders	payments	1:N	orders.id → payments.order_id
payments	refunds	1:N	payments.id → refunds.payment_id
orders	status history	1:N	orders.id → order_status_history.order_id
customers	addresses	1:N	customers.id → customer_addresses.customer_id
customers	carts	1:N	customers.id → carts.customer_id
carts	cart_items	1:N	carts.id → cart_items.cart_id
coupons	redemptions	1:N	coupons.id → coupon_redemptions.coupon_id
products	reviews	1:N	products.id → reviews.product_id

Design principle: separate mutable catalog state from immutable transaction snapshots. Do not merge the entire system into one table merely to reduce joins.

2. Catalog Schema

2.1 categories

Column	Type	Rules	Purpose
id	BIGINT	PK	Category ID

Column	Type	Rules	Purpose
name	VARCHAR(100)	UNIQUE NOT NULL	Display name
slug	VARCHAR(120)	UNIQUE NOT NULL	URL identifier
description	TEXT	NULL	Description
image_name	VARCHAR(255)	NULL	External media filename/reference
is_active	BOOLEAN	NOT NULL DEFAULT true	Public visibility
display_order	INTEGER	NOT NULL DEFAULT 0	Ordering
created_at	TIMESTAMPTZ	NOT NULL	Created
updated_at	TIMESTAMPTZ	NOT NULL	Updated

2.2 products

Column	Type	Rules	Purpose
id	BIGINT	PK	Product ID
category_id	BIGINT	FK NOT NULL	Category
product_code	VARCHAR(30)	UNIQUE NOT NULL	Internal code
name	VARCHAR(200)	NOT NULL	Product name
slug	VARCHAR(220)	UNIQUE NOT NULL	SEO URL
description	TEXT	NULL	Description
image_name	VARCHAR(255)	NULL	Primary image reference
product_type	ENUM	NOT NULL	CAKE/SNACK
is_active	BOOLEAN	NOT NULL	Public visibility
is_featured	BOOLEAN	NOT NULL	Featured flag
display_order	INTEGER	NOT NULL	Ordering
created_at	TIMESTAMPTZ	NOT NULL	Created
updated_at	TIMESTAMPTZ	NOT NULL	Updated

2.3 product_variants

Column	Type	Rules	Purpose
id	BIGINT	PK	Variant ID
product_id	BIGINT	FK NOT NULL	Parent product
name	VARCHAR(50)	NOT NULL	0.5kg, 1kg etc.
weight_value	DECIMAL(8,3)	NULL	Weight
weight_unit	VARCHAR(10)	NULL	kg/g
price	NUMERIC(10,2)	NOT NULL >= 0	Selling price
egg_type	ENUM	NOT NULL	EGG/EGGLESS
serves	VARCHAR(50)	NULL	Serving range
stock_quantity	INTEGER	NOT NULL DEFAULT 0	Optional inventory quantity
is_available	BOOLEAN	NOT NULL	Purchase availability
created_at	TIMESTAMPTZ	NOT NULL	Created
updated_at	TIMESTAMPTZ	NOT NULL	Updated

2.4 product_offers

Column	Type	Rules	Purpose
id	BIGINT	PK	Offer ID
product_id	BIGINT	FK	Owning product
buy_variant_id	BIGINT	FK	Required variant
free_variant_id	BIGINT	FK	Free variant
buy_quantity	INTEGER	>0	Buy quantity

Column	Type	Rules	Purpose
free_quantity	INTEGER	>0	Free quantity
is_active	BOOLEAN	NOT NULL	Active flag
starts_at	TIMESTAMPTZ	NULL	Start
ends_at	TIMESTAMPTZ	NULL	End
created_at	TIMESTAMPTZ	NOT NULL	Created
updated_at	TIMESTAMPTZ	NOT NULL	Updated

3. Customized Cakes & Preparation Gallery

3.1 customized_cakes

Customized cakes remain standalone predefined products. They are not an online cake-design builder. Each row has one predefined weight and one price, matching the supplied project design.

Column	Type	Rules	Purpose
id	BIGINT	PK	ID
product_code	VARCHAR(30)	UNIQUE NOT NULL	CC-001 etc.
name	VARCHAR(200)	NOT NULL	Cake name
category	VARCHAR(100)	NOT NULL	Admin-editable category
weight_value	DECIMAL(8,3)	NOT NULL	Weight
weight_unit	VARCHAR(10)	NOT NULL	kg/g
description	TEXT	NULL	Details
price	NUMERIC(10,2)	NOT NULL >=0	Price
image_name	VARCHAR(255)	NULL	Image reference
is_active	BOOLEAN	NOT NULL	Public visibility
created_at	TIMESTAMPTZ	NOT NULL	Created
updated_at	TIMESTAMPTZ	NOT NULL	Updated

3.2 preparation_gallery

Column	Type	Rules	Purpose
id	BIGINT	PK	ID
title	VARCHAR(200)	NOT NULL	Title
category	VARCHAR(100)	NOT NULL	Content category
description	TEXT	NULL	Preparation description
image_url	TEXT	NULL	Image URL
video_url	TEXT	NULL	Video URL
external_url	TEXT	NULL	Other URL
display_order	INTEGER	NOT NULL	Ordering
is_active	BOOLEAN	NOT NULL	Visibility
created_at	TIMESTAMPTZ	NOT NULL	Created
updated_at	TIMESTAMPTZ	NOT NULL	Updated

Media remains outside Neon. Use object storage/CDN for images and an appropriate video platform/storage service for videos. Store only stable references/URLs in PostgreSQL.

4. Customer & Address Schema

Guest checkout is supported. A customer account is optional. If the site later adds login, the same customer table can support account history without redesigning orders.

4.1 customers

Column	Type	Rules	Purpose
id	BIGINT	PK	Customer ID
email	VARCHAR(255)	UNIQUE NULL	Optional account email
phone	VARCHAR(30)	INDEX NOT NULL	Primary contact
name	VARCHAR(150)	NOT NULL	Name
is_active	BOOLEAN	NOT NULL	Account state
created_at	TIMESTAMPTZ	NOT NULL	Created
updated_at	TIMESTAMPTZ	NOT NULL	Updated

4.2 customer_addresses

Column	Type	Rules	Purpose
id	BIGINT	PK	Address ID
customer_id	BIGINT	FK	Owner
label	VARCHAR(50)	NULL	Home/Work/etc.
recipient_name	VARCHAR(150)	NOT NULL	Delivery recipient
phone	VARCHAR(30)	NOT NULL	Delivery phone
address_line1	TEXT	NOT NULL	Address
address_line2	TEXT	NULL	Additional address
landmark	VARCHAR(255)	NULL	Landmark
city	VARCHAR(100)	NOT NULL	City
state	VARCHAR(100)	NULL	State
pincode	VARCHAR(20)	NOT NULL	Postal code
is_default	BOOLEAN	NOT NULL	Default flag
created_at	TIMESTAMPTZ	NOT NULL	Created
updated_at	TIMESTAMPTZ	NOT NULL	Updated

Important: an order must copy the delivery address into its own columns. Never depend on a mutable customer address row for historical order delivery data.

5. Cart & Checkout Schema

5.1 carts

Column	Type	Rules	Purpose
id	BIGINT	PK	Cart ID
customer_id	BIGINT	FK NULL	Optional account
session_id	VARCHAR(128)	UNIQUE NULL	Guest browser session
status	ENUM	NOT NULL	ACTIVE/CONVERTED/ABANDONED/EXPIRED
currency	CHAR(3)	NOT NULL	INR etc.
expires_at	TIMESTAMPZ	NULL	Expiry
created_at	TIMESTAMPZ	NOT NULL	Created
updated_at	TIMESTAMPZ	NOT NULL	Updated

5.2 cart_items

Column	Type	Rules	Purpose
id	BIGINT	PK	ID
cart_id	BIGINT	FK	Cart
source_type	ENUM	NOT NULL	NORMAL/CUSTOMIZED
product_id	BIGINT	NULL FK	Normal product
product_variant_id	BIGINT	NULL FK	Normal variant
customized_cake_id	BIGINT	NULL FK	Customized cake
quantity	INTEGER	>0	Quantity
unit_price	NUMERIC(10,2)	NOT NULL	Current displayed price snapshot
created_at	TIMESTAMPZ	NOT NULL	Created
updated_at	TIMESTAMPZ	NOT NULL	Updated

The cart is not the final source of truth for price. Checkout re-reads catalog records and recalculates all totals server-side.

5.3 Cart rules

- One active cart per customer/session where practical.
- Never trust cart prices from the browser.
- Revalidate availability and promotions during checkout.
- Expired/abandoned carts can be cleaned up asynchronously.
- Do not keep payment state in the cart.

6. Production Order Schema

Column	Type	Rules	Purpose
id	BIGINT	PK	Internal ID
order_number	VARCHAR(30)	UNIQUE	Public order number
customer_id	BIGINT	NULL FK	Optional registered customer
customer_name	VARCHAR(150)	NOT NULL	Snapshot
customer_phone	VARCHAR(30)	NOT NULL	Snapshot
customer_email	VARCHAR(255)	NULL	Snapshot
delivery_recipient	VARCHAR(150)	NOT NULL	Recipient snapshot
delivery_phone	VARCHAR(30)	NOT NULL	Delivery phone
delivery_address	TEXT	NOT NULL	Address snapshot
delivery_landmark	VARCHAR(255)	NULL	Landmark

Column	Type	Rules	Purpose
delivery_city	VARCHAR(100)	NOT NULL	City
delivery_state	VARCHAR(100)	NULL	State
delivery_pincode	VARCHAR(20)	NOT NULL	Postal code
subtotal	NUMERIC(12,2)	NOT NULL >=0	Item subtotal
discount_amount	NUMERIC(12,2)	NOT NULL >=0	Discount
delivery_fee	NUMERIC(12,2)	NOT NULL >=0	Delivery fee
tax_amount	NUMERIC(12,2)	NOT NULL >=0	Tax
total_amount	NUMERIC(12,2)	NOT NULL >=0	Final total
currency	CHAR(3)	NOT NULL	INR
payment_method	ENUM	NOT NULL	ONLINE/COD
payment_status	ENUM	NOT NULL	Payment state
order_status	ENUM	NOT NULL	Fulfillment state
customer_note	TEXT	NULL	Customer note
admin_note	TEXT	NULL	Internal note
cancellation_reason	TEXT	NULL	Reason
delivery_date	DATE	NULL	Requested date
delivery_time_slot_id	BIGINT	NULL FK	Selected slot
created_at	TIMESTAMPTZ	NOT NULL	Created
updated_at	TIMESTAMPTZ	NOT NULL	Updated

7. Order Items & Order Lifecycle

7.1 order_items

Column	Type	Rules	Purpose
id	BIGINT	PK	Line ID
order_id	BIGINT	FK	Order
source_type	ENUM	NOT NULL	NORMAL/CUSTOMIZED
product_id	BIGINT	Nullable FK	Normal product
product_variant_id	BIGINT	Nullable FK	Normal variant
customized_cake_id	BIGINT	Nullable FK	Customized cake
product_name	VARCHAR(200)	NOT NULL	Historical name
variant_name	VARCHAR(100)	NULL	Historical variant
weight_value	DECIMAL(8,3)	NOT NULL	Historical weight
weight_unit	VARCHAR(10)	NOT NULL	Historical unit
unit_price	NUMERIC(12,2)	NOT NULL	Historical unit price
quantity	INTEGER	>0	Quantity
item_total	NUMERIC(12,2)	NOT NULL	Line total
image_name	VARCHAR(255)	NULL	Historical image reference
created_at	TIMESTAMP TZ	NOT NULL	Created

7.2 Order status history

	Type	Purpose	
	BIGINT PK	History ID	
	BIGINT FK	Order	
	ENUM NULL	Previous status	
	ENUM	New status	
/admin_id	BIGINT NULL FK	Admin actor	
	TEXT NULL	Reason/note	
	TIMESTAMPTZ	Created	When change occurred

7.3 Statuses

[illegible]

Do not physically delete completed orders. Keep order history. Operational corrections should be represented through status/history/refund records.

8. Razorpay Production Payment Architecture

Do not put the entire Razorpay lifecycle into orders. Orders represent the commercial transaction; payments represent payment attempts; refunds represent money returned; `webhook_events` provide an auditable gateway-event record.

8.1 payments

Column	Type	Rules	Purpose
id	BIGINT	PK	Internal payment ID
order_id	BIGINT	FK	Order
gateway	VARCHAR(30)	NOT NULL	RAZORPAY
gateway_order_id	VARCHAR(100)	INDEX	Razorpay order ID
gateway_payment_id	VARCHAR(100)	UNIQUE NULL	Razorpay payment ID
gateway_signature	VARCHAR(255)	NULL	Verified checkout signature

Column	Type	Rules	Purpose
amount	NUMERIC(12,2)	NOT NULL	Expected payment amount
amount_paid	NUMERIC(12,2)	NOT NULL	Captured/paid amount
amount_refunded	NUMERIC(12,2)	NOT NULL	Total refunded
currency	CHAR(3)	NOT NULL	INR
status	VARCHAR(40)	NOT NULL	Gateway status
method	VARCHAR(40)	NULL	card/upi/netbanking/etc.
captured	BOOLEAN	NOT NULL	Capture state
authorized_at	TIMESTAMPZ	NULL	Authorization time
captured_at	TIMESTAMPZ	NULL	Capture time
paid_at	TIMESTAMPZ	NULL	Paid time
failed_at	TIMESTAMPZ	NULL	Failure time
fee_amount	NUMERIC(12,2)	NULL	Gateway fee if available
tax_amount	NUMERIC(12,2)	NULL	Gateway tax if available
error_code	VARCHAR(100)	NULL	Gateway error
error_description	TEXT	NULL	Gateway error details
error_source	VARCHAR(100)	NULL	Gateway error source
error_step	VARCHAR(100)	NULL	Gateway error step
error_reason	VARCHAR(255)	NULL	Gateway error reason
last_synced_at	TIMESTAMPZ	NULL	Last gateway sync
created_at	TIMESTAMPZ	NOT NULL	Created
updated_at	TIMESTAMPZ	NOT NULL	Updated

9. Razorpay Refunds & Webhooks

9.1 refunds

Column	Type	Purpose
id	BIGINT PK	Internal ID
payment_id	BIGINT FK	Payment being refunded
gateway_refund_id	VARCHAR(100) UNIQUE	Razorpay refund ID
amount	NUMERIC(12,2)	Refund amount
currency	CHAR(3)	Currency
status	VARCHAR(40)	Refund status
speed_requested	VARCHAR(30)	Requested speed
speed_processed	VARCHAR(30)	Processed speed
receipt	VARCHAR(255)	Refund receipt
reference	VARCHAR(255)	Merchant reference
acquirer_reference	VARCHAR(255)	Acquirer reference if available
failure_reason	TEXT	Failure reason
created_at	TIMESTAMP TZ	Created
processed_at	TIMESTAMP TZ	Processed
failed_at	TIMESTAMP TZ	Failed

9.2 webhook_events

Column	Type	Rules	Purpose
id	BIGINT	PK	Internal ID
gateway_event_id	VARCHAR(150)	UNIQUE	Idempotency key if supplied/derived
gateway	VARCHAR(30)	NOT NULL	RAZORPAY
event_type	VARCHAR(100)	NOT NULL	payment.captured etc.
signature	VARCHAR(255)	NOT NULL	Webhook signature
payload_hash	VARCHAR(128)	INDEX	Hash for dedupe/integrity
payload	JSONB	NOT NULL	Raw event payload
processing_status	ENUM	NOT NULL	RECEIVED/PROCESSING/PROCESSED/FAILED
attempt_count	INTEGER	NOT NULL	Retry count
received_at	TIMESTAMP TZ	NOT NULL	Received
processed_at	TIMESTAMP TZ	NULL	Processed
failed_at	TIMESTAMP TZ	NULL	Failed
error_message	TEXT	NULL	Failure details

9.3 Razorpay security rules

- Verify checkout signature on the server using the exact Razorpay order/payment identifiers and secret.
- Verify webhook signature using the raw request body; do not verify a reserialized JSON body.
- Persist webhook receipt before business processing where practical.
- Make processing idempotent so retries cannot create duplicate state changes.
- Never mark an order PAID from an unverified browser callback.
- Use server-to-server verification/reconciliation for ambiguous payment states.
- Keep Razorpay secret keys and webhook secrets only in server-side environment variables.

Razorpay documentation should be treated as the authoritative source for the exact current signature and webhook verification procedure, event names, and entity fields when implementing the gateway.

10. Coupons & Promotions

Coupons are a production e-commerce recommendation beyond the originally supplied catalog/offers design. Keep catalog Buy-X-Get-Y offers separate from customer-entered coupon codes.

10.1 coupons

Column	Type	Purpose
id	BIGINT PK	Coupon ID
code	VARCHAR(50) UNIQUE	Customer-entered code
description	TEXT	Admin description
discount_type	ENUM	PERCENT/FIXED
discount_value	NUMERIC(12,2)	Discount value
minimum_order_amount	NUMERIC(12,2)	Minimum subtotal
maximum_discount_amount	NUMERIC(12,2)	Cap for percentage discount
usage_limit	INTEGER NULL	Total uses
per_customer_limit	INTEGER NULL	Uses/customer
used_count	INTEGER	Current count
starts_at	TIMESTAMPTZ	Start
ends_at	TIMESTAMPTZ	End
is_active	BOOLEAN	Active
created_at	TIMESTAMPTZ	Created
updated_at	TIMESTAMPTZ	Updated

10.2 coupon_redemptions

Column	Type	Purpose
id	BIGINT PK	ID
coupon_id	BIGINT FK	Coupon
order_id	BIGINT FK	Order
customer_id	BIGINT NULL FK	Customer if known
discount_amount	NUMERIC(12,2)	Actual discount
created_at	TIMESTAMPTZ	Redemption time

11. Delivery & Availability

11.1 delivery_zones

Column	Type	Purpose
id	BIGINT PK	Zone ID
name	VARCHAR(100)	Zone name
pincode_pattern	VARCHAR(255)	Supported postal pattern/list strategy
delivery_fee	NUMERIC(12,2)	Default fee
minimum_order_amount	NUMERIC(12,2)	Minimum
is_active	BOOLEAN	Availability
created_at	TIMESTAMPTZ	Created
updated_at	TIMESTAMPTZ	Updated

11.2 delivery_slots

Column	Type	Purpose
id	BIGINT PK	Slot ID
name	VARCHAR(100)	Morning/Evening
start_time	TIME	Start
end_time	TIME	End
max_orders	INTEGER	Capacity
is_active	BOOLEAN	Availability

If the shop does not need delivery capacity or zones initially, these tables can be disabled in the first release. They are included because a production e-commerce cake site commonly needs date/time delivery constraints.

12. Inventory

For made-to-order cakes, inventory can be intentionally lightweight. The recommended design tracks variant availability and optionally an inventory ledger when ingredient/production capacity management becomes necessary.

12.1 inventory_ledger

Column	Type	Purpose
id	BIGINT PK	Ledger ID
product_variant_id	BIGINT FK	Variant
change_quantity	INTEGER	Positive/negative change
reason	VARCHAR(100)	SALE/RESTOCK/ADJUSTMENT/CANCEL
reference_order_id	BIGINT NULL FK	Related order
note	TEXT	Reason
created_at	TIMESTAMPTZ	When changed

Do not implement aggressive stock reservation unless the business actually has limited stock. For made-to-order cakes, availability may be capacity-based rather than inventory-count based.

13. Reviews

Column	Type	Rules	Purpose
id	BIGINT	PK	Review ID
product_id	BIGINT	FK	Normal product
customer_id	BIGINT NULL FK	Customer	
order_id	BIGINT NULL FK	Verified purchase source	

Column	Type	Rules	Purpose
rating	SMALLINT	1-5	Rating
title	VARCHAR(150)	NULL	Title
body	TEXT	NULL	Review
status	ENUM	PENDING/APPROVED/REJECTED	Moderation
created_at	TIMESTAMPTZ	Created	
updated_at	TIMESTAMPTZ	Updated	

If reviews are not part of the first release, omit them rather than creating unused complexity. The table is included here as a production-ready optional module.

14. Admin, Audit & Security Data

14.1 admins

Column	Type	Purpose
id	BIGINT PK	Admin ID
email	VARCHAR(255) UNIQUE	Login identity
name	VARCHAR(150)	Display name
role	ENUM	SUPER_ADMIN/ADMIN/STAFF
is_active	BOOLEAN	Account state
last_login_at	TIMESTAMPTZ	Last login
created_at	TIMESTAMPTZ	Created
updated_at	TIMESTAMPTZ	Updated

14.2 audit_logs

Column	Type	Purpose
id	BIGINT PK	Log ID
admin_id	BIGINT NULL FK	Actor
action	VARCHAR(100)	CREATE/UPDATE/DELETE/STATUS/etc.
entity_type	VARCHAR(100)	Affected entity
entity_id	BIGINT NULL	Affected ID
before_data	JSONB NULL	Before snapshot
after_data	JSONB NULL	After snapshot
ip_address	INET NULL	Source IP if appropriate
user_agent	TEXT NULL	Client info
created_at	TIMESTAMPTZ	Time

Never store passwords in plain text. Prefer a managed identity provider for admin authentication where possible; the database admin table can hold application-level admin metadata and authorization roles.

15. Final Enumerations

```
ProductType:      CAKE | SNACK
EggType:          EGG | EGGLESS
OrderSourceType:  NORMAL | CUSTOMIZED
PaymentMethod:    ONLINE | COD

PaymentStatus:
  PENDING | AUTHORIZED | PAID | FAILED | REFUNDED | PARTIALLY_REFUNDED

OrderStatus:
  PENDING | CONFIRMED | PREPARING | READY |
  OUT_FOR_DELIVERY | DELIVERED | CANCELLED

CartStatus:
  ACTIVE | CONVERTED | ABANDONED | EXPIRED

CouponDiscountType:
  PERCENT | FIXED

ReviewStatus:
  PENDING | APPROVED | REJECTED

WebhookProcessingStatus:
  RECEIVED | PROCESSING | PROCESSED | FAILED
```

16. Database Integrity Rules

- All money fields use NUMERIC/Decimal, never FLOAT.
- All IDs use BIGINT/Prisma BigInt consistently.
- All foreign keys must reference valid parent records.
- Order items use explicit source references and a source-type CHECK constraint.
- Offer buy/free variants must belong to the same product as product_offers.product_id.
- Order totals are calculated on the server and persisted.
- Order item snapshots are immutable after checkout except through controlled correction procedures.
- Payment state is changed only from verified gateway/server events.
- Refund totals may never exceed the captured/paid amount.
- Coupon usage is incremented transactionally to prevent race-condition overuse.
- Delivery slot capacity is checked transactionally if capacity is enforced.
- Completed orders are not physically deleted.
- Admin mutations are audited.

16.1 Coupon race-condition pattern

```
BEGIN
  SELECT coupon
  FROM coupons
  WHERE id = ...
  FOR UPDATE;

  validate usage limits;

  INSERT coupon_redemptions (...);

  UPDATE coupons
  SET used_count = used_count + 1;

COMMIT;
```

16.2 Delivery capacity race-condition pattern

```
BEGIN
  lock/check the selected slot or use an atomic capacity update;
  verify current order count < max_orders;
  create order;
COMMIT;
```

Only implement row locking/capacity controls if delivery slots are actually capacity-limited.

17. Production API Contract

Method	Endpoint	Purpose
GET	/api/categories	Active categories
GET	/api/products	Catalog/filter/search
GET	/api/products/:slug	Product + variants + offers
GET	/api/customized-cakes	Customized catalog
GET	/api/gallery	Preparation content
POST	/api/cart	Create/update guest cart
GET	/api/cart/:id	Read cart
POST	/api/orders	Create order
POST	/api/payments/create	Create Razorpay order/payment flow
POST	/api/payments/verify	Verify checkout result
POST	/api/payments/webhook	Razorpay webhook receiver
POST	/api/refunds	Admin refund operation

Method	Endpoint	Purpose
GET	/api/orders/:orderNumber	Order lookup
GET	/api/admin/orders	Admin order list
PATCH	/api/admin/orders/:id/status	Update order status
POST	/api/admin/products	Create product
PATCH	/api/admin/products/:id	Update product
POST	/api/admin/customized-cakes	Create customized cake
POST	/api/admin/gallery	Create gallery content

17.1 API security rules

- Never expose DATABASE_URL.
- Never trust price/discount/total values from the browser.
- Validate every ID and quantity.
- Use transactions for order creation.
- Protect all admin endpoints.
- Rate-limit checkout, payment verification, login, coupon validation, and webhook endpoints as appropriate.
- Return generic errors to customers; keep sensitive details in server logs.

18. End-to-End Checkout & Razorpay Flow

```
CUSTOMER
↓
Browse catalog
↓
Cart
↓
Checkout details
↓
SERVER re-loads catalog + validates prices/offers/availability
↓
SERVER calculates subtotal/discount/delivery/tax/total
↓
CREATE PENDING ORDER + ORDER ITEMS (transaction)
↓
CREATE RAZORPAY ORDER
↓
Store gateway_order_id in payments
↓
Customer pays
↓
Razorpay returns payment_id + signature
↓
SERVER verifies signature
↓
Store gateway_payment_id/signature
↓
Confirm/capture according to configured Razorpay flow
↓
Webhook received
↓
Verify raw-body webhook signature
↓
Idempotently update payment + order
↓
PAID / CONFIRMED
```

18.1 Payment states vs order states

Do not treat `payment_status` and `order_status` as the same field. A payment can be `PAID` while the cake is still `PREPARING`. A `COD` order can be `CONFIRMED` while `payment_status` remains `PENDING`.

18.2 Refund flow

```
ADMIN requests refund
↓
Server checks order/payment/refundable amount
↓
Razorpay refund request
↓
Store refunds row
↓
Webhook / gateway status update
↓
Update refund status
↓
Recalculate payment.amount_refunded
↓
Set payment_status = REFUNDED or PARTIALLY_REFUNDED
```

19. Prisma Production Model Strategy

Use Prisma as the application's schema/model layer and Prisma Migrate as the controlled migration mechanism. Review generated SQL for important constraints, composite foreign keys, indexes, enums, and database-specific behavior before production.

Development:
npx prisma validate
npx prisma generate
npx prisma migrate dev --name <change>

Staging:
npx prisma migrate deploy
npx prisma generate

Production:
npx prisma migrate deploy
npx prisma generate

19.1 Type mapping

Business data	PostgreSQL	Prisma
ID	BIGINT	BigInt
Money	NUMERIC(12,2)	Decimal
Weight	NUMERIC(8,3)	Decimal
Timestamp	TIMESTAMPZ	DateTime
Date	DATE	DateTime @db.Date
Boolean	BOOLEAN	Boolean
Flexible gateway payload	JSONB	Json

19.2 updated_at

Prisma's @updatedAt updates timestamps for Prisma writes. If raw SQL or another writer modifies the database, use a PostgreSQL trigger or ensure the writer explicitly updates updated_at. Do not assume DEFAULT NOW() means the timestamp automatically changes on every UPDATE.

20. Neon Production Strategy

DEVELOPMENT
Neon development branch
↓
STAGING
isolated branch/project
↓
PRODUCTION
dedicated production branch/project

APP
↓
Prisma
↓
Neon pooled connection
↓
PostgreSQL

- Use separate development/staging/production database URLs.
- Use pooled connections for application traffic where appropriate.
- Keep migrations controlled through Prisma Migrate.
- Do not store images/videos in PostgreSQL.
- Monitor compute, storage, egress, query latency, and connection behavior.
- Do not add indexes without a query reason.
- Avoid N+1 queries.

Neon plan limits and pricing can change. Check the current Neon console/pricing page before launch rather than hard-coding today's limits into application logic.

21. Index Strategy

Table	Recommended indexes	Why
categories	slug UNIQUE; is_active/display_order	Public category listing
products	category_id; product_type; slug UNIQUE	Catalog/filter/URL
product_variants	product_id; UNIQUE(product_id,name)	Variant lookup
product_offers	product_id; buy_variant_id; free_variant_id	Offer lookup
customized_cakes	category; name; product_code UNIQUE	Filtering/search
preparation_gallery	category; display_order; is_active	Gallery
customers	phone; email UNIQUE where applicable	Lookup/account
customer_addresses	customer_id	Address list
carts	session_id; customer_id; status	Cart lookup
cart_items	cart_id	Cart retrieval
orders	order_number UNIQUE; customer_phone; customer_id; order_status; payment_method; created_at; delivery_date	Admin/customer operations
order_items	order_id; product_variant_id; customized_cake_id	Order loading
payments	order_id; gateway_order_id; gateway_payment_id UNIQUE	Payment lookup
refunds	payment_id; gateway_refund_id UNIQUE	Refund lookup
webhook_events	gateway_event_id UNIQUE; payload_hash	Idempotency/audit
coupon_redemptions	coupon_id; order_id; customer_id	Usage
reviews	product_id; status	Product reviews
audit_logs	admin_id; entity_type/entity_id; created_at	Admin audit

Indexes should be validated against actual query plans. Extra indexes increase write and storage overhead.

22. Core PostgreSQL DDL — Production Baseline

```
CREATE TABLE payments (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  order_id BIGINT NOT NULL REFERENCES orders(id) ON DELETE RESTRICT,
  gateway VARCHAR(30) NOT NULL,
  gateway_order_id VARCHAR(100),
  gateway_payment_id VARCHAR(100) UNIQUE,
  gateway_signature VARCHAR(255),
  amount NUMERIC(12,2) NOT NULL CHECK (amount >= 0),
  amount_paid NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (amount_paid >= 0),
  amount_refunded NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (amount_refunded >= 0),
  currency CHAR(3) NOT NULL DEFAULT 'INR',
  status VARCHAR(40) NOT NULL,
  method VARCHAR(40),
  captured BOOLEAN NOT NULL DEFAULT FALSE,
  authorized_at TIMESTAMPTZ,
  captured_at TIMESTAMPTZ,
  paid_at TIMESTAMPTZ,
  failed_at TIMESTAMPTZ,
  fee_amount NUMERIC(12,2),
  tax_amount NUMERIC(12,2),
  error_code VARCHAR(100),
  error_description TEXT,
  error_source VARCHAR(100),
  error_step VARCHAR(100),
  error_reason VARCHAR(255),
  last_synced_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE INDEX idx_payments_order ON payments(order_id);
CREATE INDEX idx_payments_gateway_order ON payments(gateway_order_id);
CREATE UNIQUE INDEX uq_payments_gateway_payment
  ON payments(gateway_payment_id)
  WHERE gateway_payment_id IS NOT NULL;
```

```
CREATE TABLE refunds (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  payment_id BIGINT NOT NULL REFERENCES payments(id) ON DELETE RESTRICT,
  gateway_refund_id VARCHAR(100) UNIQUE,
  amount NUMERIC(12,2) NOT NULL CHECK (amount > 0),
  currency CHAR(3) NOT NULL DEFAULT 'INR',
  status VARCHAR(40) NOT NULL,
  speed_requested VARCHAR(30),
```

```

speed_processed VARCHAR(30),
receipt VARCHAR(255),
reference VARCHAR(255),
acquirer_reference VARCHAR(255),
failure_reason TEXT,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
processed_at TIMESTAMPTZ,
failed_at TIMESTAMPTZ
);

CREATE TABLE webhook_events (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  gateway_event_id VARCHAR(150) NOT NULL UNIQUE,
  gateway VARCHAR(30) NOT NULL,
  event_type VARCHAR(100) NOT NULL,
  signature VARCHAR(255) NOT NULL,
  payload_hash VARCHAR(128),
  payload JSONB NOT NULL,
  processing_status VARCHAR(30) NOT NULL DEFAULT 'RECEIVED',
  attempt_count INTEGER NOT NULL DEFAULT 0,
  received_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  processed_at TIMESTAMPTZ,
  failed_at TIMESTAMPTZ,
  error_message TEXT
);

```

23. Migration, Backup & Recovery

23.1 Recovery rules

- Have a documented recovery/backup strategy before launch.
- Never treat migrations as backups.
- Test restoration procedures periodically.
- Keep production database credentials in a secrets manager/platform secret store.
- For destructive migrations, use additive/backfill/remove phases where practical.

24. Security Checklist

- TLS/SSL database connections.
- Server-only DATABASE_URL.
- Server-only Razorpay secret key and webhook secret.
- Admin authentication + authorization.
- Rate limiting on sensitive endpoints.
- Input validation and output encoding.
- Webhook signature verification using raw request body.
- Idempotent payment/webhook processing.
- No payment card details stored in the database.
- No plaintext passwords.
- Audit admin mutations.
- Avoid sensitive information in application logs.
- Use security headers and HTTPS at the web layer.

25. Production Test Plan

Area	Tests
Catalog	Category/product/variant filtering; inactive items hidden; price correct
Customized	Correct category/name/weight/description/price/image
Cart	Add/update/remove; quantity validation; guest session
Checkout	Server-side repricing; invalid item rejected; transaction rollback
Orders	Snapshot preserved after product price/name changes
Razorpay	Order creation; signature verification; payment state; webhook retry/idempotency
Refunds	Full refund; partial refund; duplicate webhook
Coupons	Validity; limits; race-condition protection
Delivery	Pincode/zone; slot capacity if enabled
Admin	Authorization; status transitions; audit log
Database	FKs; checks; indexes; migration up/down strategy
Recovery	Restore procedure and production rollback plan

26. Final Production Launch Checklist

Database

- All migrations reviewed

- Foreign keys/checks validated
- Indexes validated
- Production connection tested

Neon

- Production project/branch configured
- Pooling configured
- Secrets configured
- Usage monitoring configured

Catalog

- Products/variants seeded
- Customized cakes seeded
- Gallery content configured
- Media CDN/object storage configured

Checkout

- Guest checkout tested
- Server-side price calculation tested
- Cart expiry behavior tested
- Order transaction tested

Razorpay

- Production key configured
- Webhook secret configured
- Signature verification tested
- Webhook idempotency tested
- Payment reconciliation tested
- Refund flow tested

Orders

- Order status workflow tested
- Order history immutable
- Admin status updates audited
- Delivery fields validated

Security

- Admin auth enabled
- Rate limiting configured
- No secrets in client bundle
- Logs scrubbed

Operations

- Monitoring enabled
- Error alerts enabled
- Recovery procedure documented
- Rollback procedure documented

27. Final Production Schema — One-Page Reference

CATALOG
categories
 products
 product_variants
 product_offers

customized_cakes
preparation_gallery

CUSTOMER
customers
 customer_addresses

CART
carts
 cart_items

ORDERS
orders
 order_items
 order_status_history

PAYMENTS
orders
 payments
 refunds

webhook_events

PROMOTIONS
coupons
 coupon_redemptions

OPERATIONS
delivery_zones
delivery_slots
inventory_ledger

TRUST
reviews
admins
audit_logs

Component	Final production decision
Normal catalog	categories → products → variants/offers
Customized cakes	Standalone predefined catalog
Preparation media	Standalone gallery; external media
Customers	Optional account + guest checkout
Cart	Server-revalidated cart
Orders	Historical snapshot + status history
Payments	Dedicated Razorpay payment records
Refunds	Dedicated refund records; partial/full supported
Webhooks	Persisted + signature verified + idempotent
Coupons	Dedicated code + redemption records
Delivery	Zones/slots optional but production-ready
Inventory	Lightweight variant stock + optional ledger
Reviews	Moderated, optional
Admin	Protected roles + audit logs
Money	NUMERIC/Decimal
IDs	BIGINT/BigInt
Schema changes	Prisma Migrate
Media	Outside Neon
Deletion	No routine deletion of completed orders

Final verdict: This is the production-ready target architecture for the e-commerce cake website. The modules that are not required for the first release (reviews, inventory ledger, delivery capacity, coupons, customer accounts) can be left inactive, but their boundaries are defined so the system can grow without redesigning

28. Source Basis & Production Expansion

The supplied project documents establish the original normal product catalog, customized-cake catalog, preparation gallery, orders, order items, money rules, media-reference strategy, and historical snapshots. This document preserves those core decisions and expands the design for production e-commerce requirements such as customer/cart handling, Razorpay payment lifecycle, refunds, webhook idempotency, coupons, delivery, reviews, inventory, admin roles, and audit logging.

Customized cakes remain one-row purchasable predefined products as specified in the supplied customized-cake schema. The preparation gallery remains separate from product imagery as specified in the supplied gallery design. Order items retain historical snapshots as specified in the supplied order architecture.

External gateway implementation details must be checked against the current Razorpay API documentation immediately before production deployment because gateway entity fields, webhook events, and verification requirements can evolve.

29. Implementation Order

30. Final Recommendation

Use this document as the master database specification. Do not try to reduce Neon CPU by collapsing the schema into one giant table. The correct optimization path is: relational integrity → good indexes → efficient Prisma queries → connection pooling → external media storage → caching for mostly-static catalog data → measurement and scaling when real traffic requires it.

Production target: Neon PostgreSQL + Prisma + secure server-side APIs + Razorpay + external media storage/CDN + protected admin panel + transactional checkout + immutable order snapshots + idempotent payment/webhook processing.